

Gestión de Inventario del Departamento de Sanidad

Proyecto Intermodular

Curso 2025/2026 de 2ºDAW

IES Leonardo Da Vinci

Mateus Leandro Barros
Francisco Javier López Bautista

Sumario

Introducción al proyecto.....	3
Contexto.....	3
Mejoras solicitadas: nuestro trabajo.....	4
Descripción y objetivos.....	4
Tecnologías empleadas.....	5
Desarrollo.....	6
Enlace al repositorio de GitHub.....	6
Distribución de responsabilidades.....	6
Problemas generales detectados y corregidos.....	6
Rediseño de la bases de datos.....	8
Problemas identificados y soluciones aplicadas.....	8
Modelo Entidad-Relación (MER) resultante.....	10
Modelo Relacional (MR) resultante.....	10
Mejoras principales propuestas: resultados.....	12
Instrucciones de despliegue.....	12
Entorno local (Windows).....	12
Entorno local con Ngrok (Windows).....	16
Entorno de producción.....	17
Observaciones y mejoras a futuro.....	17
Conclusiones.....	18
Bibliografía y recursos utilizados.....	18

Introducción al proyecto

Contexto

Nuestro grupo mejorará una aplicación web existente que fue encargada por la jefa del Departamento de Sanidad del instituto, la cuál fue desarrollada desde cero en Laravel y presentada como Proyecto Intermodular por un grupo de alumnos el pasado curso 2024/2025. Los usuarios de esta aplicación son los alumnos, profesores y administradores de dicho departamento

Antes de su desarrollo, el Departamento de Sanidad del instituto gestionaba su inventario sanitario de forma manual, lo que dificultaba llevar un control eficiente de los materiales almacenados, su estado y los préstamos realizados

La aplicación tiene como función y objetivo permitir al Departamento de Sanidad gestionar de forma eficiente, organizada, actualizada y accesible su inventario de material sanitario, mejorando el control de los materiales en reserva y uso y el seguimiento de los movimientos de entrada y salida, minimizando así los errores del control manual como la pérdida y/o descontrol de material

Incluye las siguientes funcionalidades y características:

- Registro y listado en tiempo real de material sanitario
- Control de salidas y entradas de material, incluyendo su préstamo y devolución
- Gestión del stock, diferenciando entre material en uso (activo) y material en reserva (pasivo)
- Autenticación de usuarios, sistema de notificaciones, etc
- Uso diferenciado según el rol de usuario:
 - **Alumno:** puede mencionar los materiales que ha usado registrando sus prácticas (sin afectar al stock directamente) y visualizar su historial de prácticas realizadas y la totalidad de los materiales en uso que hay. Son quienes usan el material
 - **Profesor:** puede ver las prácticas realizadas por sus alumnos y la totalidad de los materiales en uso que hay. También puede gestionar el almacenamiento de material restando stock únicamente de los materiales en uso (material fungible o estropeado de las prácticas)
 - **Administrador:** tiene control completo sobre la gestión de usuarios y materiales, incluyendo altas, bajas, reposiciones y visualización de modificaciones y eventos mediante historial y notificaciones

Mejoras solicitadas: nuestro trabajo

Tras un periodo de uso de la aplicación, el cliente ha regresado para solicitar un conjunto de mejoras que considera necesarias. Estas serán abordadas por nuestro grupo en este curso 2025/2026 como parte del Proyecto Intermodular:

- **Accesibilidad QR:** Actualmente, cuando el Departamento de Sanidad debe reponer físicamente el inventario, los responsables tienen que desplazarse al almacén para realizar la reposición y, posteriormente, acudir a una estación de trabajo para acceder a la aplicación como administradores. Una vez allí, deben buscar en el almacenamiento cada material involucrado uno a uno y actualizar su stock. Sin embargo, este proceso no les resulta cómodo, ya que preferirían poder gestionarlo desde el propio almacén y acceder a la página de gestión de cada material involucrado de forma totalmente directa, con el fin de agilizar y optimizar el trabajo. Para ello, han propuesto asociar la página de gestión de cada material en un almacén a un código QR que se pegará físicamente en su contenedor correspondiente dentro del almacén. De este modo, los administradores podrán escanear cada contenedor desde sus teléfonos y ser redirigidos automáticamente a la aplicación web, donde, tras iniciar sesión como administradores, accederán directamente a la página de gestión del material correspondiente
- **Optimización de la gestión de material en el panel de administración:** En el panel de administración, el alta de un nuevo material se realiza mediante un único formulario que recoge tanto su información general como el stock inicial y los lugares de almacenamiento (Odontología y/o CAE, junto con su localización exacta dentro de ellos). No obstante, una vez registrado, la gestión del material se divide en dos secciones diferenciadas: por un lado, la gestión del material, destinada a actualizar la información general, y por otro, la gestión de almacenamiento, enfocada en la actualización de stock y localización. El cliente considera que esta separación fragmenta la gestión del material sin una justificación clara, ya que durante el registro toda la información se introduce de forma unificada. Además, el listado del material queda duplicado en ambas secciones, teniendo que repetir búsquedas para gestionar un mismo material. Por ello, propone valorar la integración de ambas secciones en una sola, donde se facilite la gestión del material en un único lugar

Descripción y objetivos

Tras analizar las mejoras propuestas por el cliente para el panel de administración y concebir un diseño, nos hemos propuesto los siguientes objetivos para cumplirlas:

- Desarrollar una nueva sección "Códigos QR", que listará todas las imágenes de los códigos QR y facilitará su descarga y/o impresión. Estos códigos QR se generarán en el alta de cada material
- Fusionar las secciones "Gestionar Materiales" (edición de la información general de los

materiales) y "Gestionar Almacenamiento" (gestión de stock y ubicación de almacenamiento de los materiales en sus almacenes) en una nueva "Gestionar Material". Para esta nueva sección, revisando la propuesta con el cliente, hemos concebido el siguiente diseño:

- `/material/update`: listado de todos los materiales
- `/material/update/edit/{material_id}`: acceso manual por navegación a la vista de edición: se podrá editar tanto la información general del material como su almacenamiento **en todos los almacenes**
- `/material/update/edit/{material_id}/almacen/{storage}`: acceso vía QR a la vista de edición: se podrá editar tanto la información general del material como su almacenamiento **en un almacén en específico**. Las URL que sigan este patrón, para cada combinación de material y almacén, serán representados por los códigos QR
- En combinación, el nuevo flujo funcionaría de la siguiente forma: los administradores descargan, imprimen y pegan en los almacenes los códigos QR. Posteriormente, quienes gestionen físicamente el almacén, podrán escanear esos códigos QR con su teléfono para ser redirigidos a la página de gestión descrita anteriormente
- Corregir los problemas en la aplicación que identifiquemos y/o que puedan interponerse en el desarrollo de las mejoras (documentados a continuación)

En definitiva, el objetivo principal es optimizar para los administradores la gestión del material sanitario

Tecnologías empleadas

Lenguajes y frameworks para el desarrollo

- **Backend**: PHP con Laravel
- **Frontend**: JavaScript y CSS, en combinación con el lenguaje de vistas de Laravel (Blade)

Herramientas

- **Composer**: Gestor de dependencias estándar de PHP

Entorno local y despliegue

- **Wamp**: para el servidor MySQL
- **Ngrok**: para publicar la aplicación en Internet provisionalmente desde el entorno local, con el fin de demostrar los avances al cliente

Desarrollo

Enlace al repositorio de GitHub

Para trabajar en el proyecto, hemos realizado una bifurcación del repositorio original. Además de poder trabajar de forma independiente con una copia exacta del repositorio, esto nos permite optar a la contribución de nuestros cambios al repositorio original si así fuese requerido

Original: https://github.com/Blanco-Joel/Inventario_Sanidad

Bifurcación: https://github.com/franciscolopez18n-ops/Inventario_Sanidad

Distribución de responsabilidades

Mateus Leandro: backend (PHP), Blade, modelado, documentación, integración

Francisco Javier López: despliegue en producción, Blade, JavaScript, CSS

Problemas generales detectados y corregidos

Durante el desarrollo de las mejoras planteadas, hemos encontrado y corregido muchos problemas, los cuáles están documentados en su totalidad a lo largo de la historia de nuestro repositorio. Documentamos a continuación los más importantes

- **En el Alta de Material (panel de administración)**
 - El envío del formulario no devolvía respuesta de ningún tipo, ni cuando fallaba ni cuando tenía éxito. Tras identificar y corregir el problema en el código, nos hemos percatado de que los mensajes flash estaban muy mal mantenidos en toda la aplicación, con fallos como mensajes perdidos escritos por el backend que nunca se mostraban en la vista correspondiente, repetición constante del mismo patrón de código en todas las vistas, inconsistencias en los tipos de mensajes, recuperación duplicada de mensajes por parte de Blade y JavaScript, código innecesario o aislado, etc. Para solucionar este problema hemos decidido organizar y mejorar los mensajes flash y su mantenimiento en general con un componente de Blade que centraliza y reutiliza la lógica de mostrado en las vistas y una clase de constantes (alternativa a un Enum para la versión PHP del proyecto) para los tipos de mensajes, permitiendo la inyección de mensajes desde JavaScript gracias a un contenedor interno. Este componente se implementa una sola vez en el layout de la página (vista plantilla)
 - Elimina el campo oculto enviado en el formulario, destinado a almacenar los datos JSON de la cookie. Es una solución inapropiada y redundante para que el backend lea los datos de la cookie, pues esta se envía automáticamente al servidor. La cookie ahora es leída directamente en el backend
 - Añade una capa de validación de seguridad en el backend para los datos de la

- cesta, con el objetivo de que el frontend no sea el único responsable de la validación, ya que un usuario podría manipularlo (o bien la cookie del carrito), permitiéndole crear potenciales inconsistencias en la base de datos contrarias a la lógica de negocio. Esto es más por seguir buenas prácticas de seguridad
- Valida con JavaScript que el tamaño de la cookie no supere los 4KB, evitando errores silenciosos como que el material no se guarde realmente en el carrito
 - Corrige el problema de que tras haber eliminado un material de la cesta, este igualmente se pueda dar de alta como si no hubiera sido borrado realmente
 - Identifica un problema de diseño en la tabla `storages` (documentado a continuación)
 - Elimina el soporte a imágenes SVG y GIF: no son tan apropiados como imagen de un material
 - Elimina algunas redundancias e implementaciones innecesarias (como la recuperación de errores y valores para los campos del formulario, los cuales no eran guardados por el backend en primer lugar porque ni siquiera era necesario)
 - Varios de estos problemas también los comparte el formulario de **registro de actividad**, los cuales fueron igualmente corregidos
- Sube al repositorio una imagen fallback `no_image.jpg` para los materiales, la cuál no estaba (pero debía). También corrige el problema de que los materiales iniciales tuvieran asociada una imagen que no existe
 - Elimina query params innecesarios (`_token` y `material_id`) al modificar la información general de un material en específico
 - Corrección de la gestión de sesión y protección de rutas
 - Una parte de la lógica de los códigos QR requiere que, al escanear uno sin sesión iniciada, el usuario sea redirigido al login y, tras autenticarse, llegue exactamente a donde pretendía. Esto no funcionaba: el flujo quedaba interrumpido y el usuario acababa en la página de bienvenida sin importar el origen
 - La solución era usar el método `intended()` de Laravel, pero para que funcione correctamente todas las rutas deben estar protegidas con el middleware `auth` nativo. Al auditar las rutas para aplicar esta corrección, se identificaron varios problemas adicionales:
 - Rutas de paneles accesibles por otros roles o incluso con la sesión cerrada, mostrando vistas incompletas o disfuncionales
 - Organización deficiente de rutas
 - La ruta raíz no detectaba correctamente que la sesión ya estaba iniciada, mostrando el formulario de login a los usuarios autenticados
 - Los middlewares de control de rol eran redundantes y estaban mal escritos
 - Para corregirlo, se realizó una reforma completa:
 - Se sustituyeron las cookies de sesión por el sistema `Auth` de Laravel, que gestiona la sesión de forma automática, segura y sin reinventar la rueda. La duración de sesión de 1 día con persistencia tras cerrar el navegador se

mantiene tal como se diseñó

- Se protegieron todas las rutas con `auth` y se reescribieron los middlewares de rol
- Se integró `intended()` para garantizar que el flujo de redirección funcione correctamente en todos los escenarios, incluyendo el acceso vía QR
- Se añadió una comprobación en el formulario de login para redirigir al usuario autenticado directamente al welcome si intenta acceder a la ruta raíz con la sesión ya iniciada
- Corrección de límites mínimos en validaciones: Se han corregido los límites mínimos en todas las validaciones de la aplicación relacionadas con el almacenamiento, ya que existían inconsistencias entre formularios. El criterio establecido es el siguiente: las unidades y unidades mínimas admiten el valor 0, mientras que armario, balda y cajón deben ser estrictamente mayores que 0
- Resuelve problemas importantes en el formulario de actualización de almacenamiento:
 - En el modo de reposición en la gestión de almacenamiento, comprobaciones erróneas y registro de “0 unidades modificadas” en la tabla `modifications` si se actualizaba la ubicación en lugar del stock, lo que no informa de un cambio real
 - Diseño nada intuitivo de la interfaz:
 - La página de modificación de almacenamiento contaba con dos modos de operación poco intuitivos que se activaban mediante un checkbox:
 - **Modo de reposición:** se seleccionaba el checkbox "Actualizar solamente reserva" e introducía el nuevo total de unidades en reserva, sin afectar al stock de uso. De esta forma es como se incrementan las existencias de un material en un almacén
 - **Modo de distribución:** sin el checkbox, modificar las unidades de un almacenamiento ajustaba automáticamente el otro para compensar, sin permitir modificar ambos a la vez. De esta forma, el stock total no cambia, sino que se distribuye entre ambos almacenamientos
 - Esta mecánica resultaba muy confusa para el usuario, que debía entender el funcionamiento interno para operar correctamente. Por ello, se ha llevado a cabo un rediseño completo que se presentará a continuación en este documento

Rediseño de la bases de datos

Problemas identificados y soluciones aplicadas

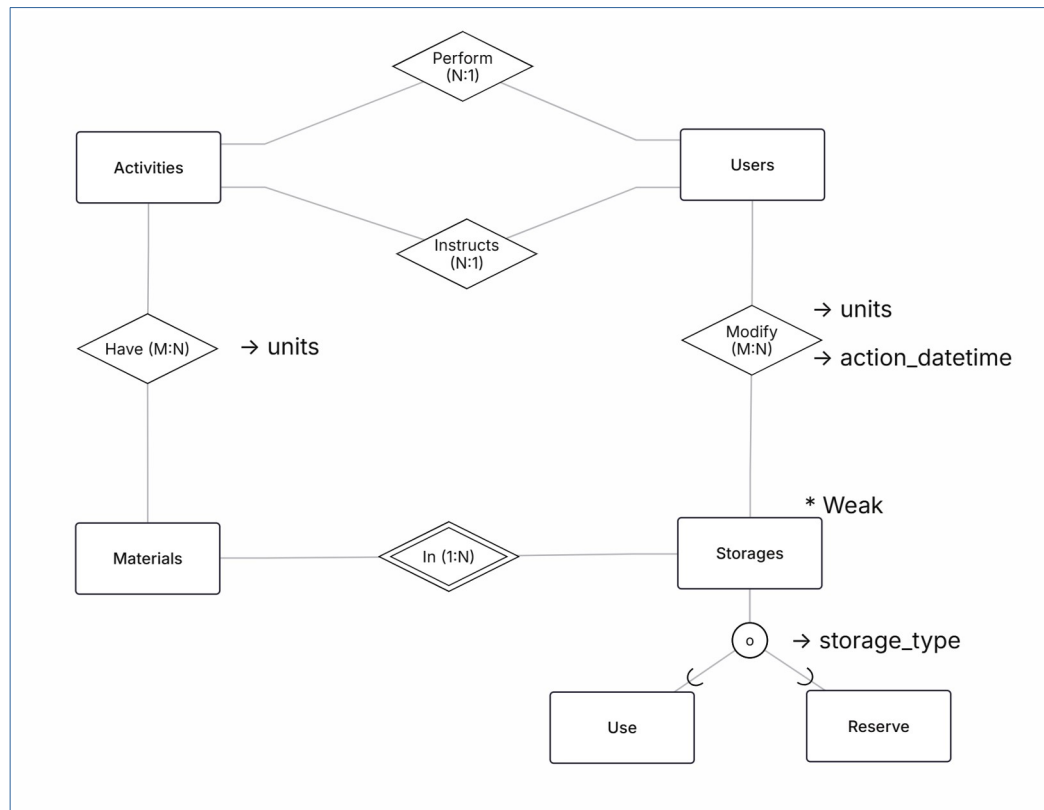
- Tabla `storages`: Esta tabla fue objeto de una revisión y normalización progresiva

motivada por varios problemas identificados

- **Señales de alerta iniciales**
 - La tabla presentaba desde el principio indicios de un diseño mejorable. Las columnas de ubicación (armario, balda, cajón) cambiaban de semántica según la modalidad de almacenamiento: en uso, el armario era un identificador numérico y el cajón tenía valor; en reserva, el armario era un identificador textual y el cajón no aplicaba. Esto generaba columnas con tipos implícitamente distintos según el contexto y nulls que expresaban inaplicabilidad estructural en lugar de ausencia de dato. Como consecuencia, la aplicación debía asumir lógica implícita para manejar estos casos, complicando la validación de formularios y el mantenimiento, pues la tabla no era flexible a posibles cambios futuros que aumentaran las diferencias entre ambas modalidades de almacenamiento. No obstante, tras un análisis formal, se comprobó que la tabla no violaba ninguna forma normal, por lo que el problema era de diseño semántico y no de normalización estrictamente
- **La normalización se vuelve necesaria**
 - El punto de inflexión llegó al implementar el atributo `qr_path`, que debía asociarse a cada combinación de material y almacén, con independencia de la modalidad. La clave primaria de `storages` estaba compuesta por `(material_id, storage, storage_type)`, pero `qr_path` solo dependía de `(material_id, storage)`, es decir, de una parte de la clave. Esto constituye una dependencia parcial y una violación clara de la 2FN, que habría resultado en que el QR se duplicara para cada modalidad del mismo material en el mismo almacén
- **Solución adoptada**
 - Tras analizar el problema con detenimiento, se optó por convertir `storages` en una jerarquía total y solapada en el modelo entidad-relación. El resultado en el modelo relacional es el siguiente:
 - `storages`: supertipo, con los atributos comunes `(material_id, storage, qr_path)`
 - `storage_use` y `storage_reserve`: subtipos con sus atributos específicos de ubicación y stock
 - `storage_assignments`: tabla de la jerarquía que recoge el discriminante `storage_type`, con clave primaria compuesta por `(material_id, storage, storage_type)`
 - Esta última tabla se relaciona con `modifications`, donde el tipo de almacenamiento adquiere importancia estructural como referencia a qué modalidad fue modificada, garantizando la integridad referencial
 - En el código, los modelos de los subtipos implementan la interfaz `StockStorage`, que establece un contrato común y permite el polimorfismo sin necesidad de discriminar el tipo explícitamente en la capa de aplicación
- **Tabla `activities`**: Esta tabla presentaba un problema menor: la columna

`teacher_id` no tenía agregada la restricción de clave extranjera. Esto significaba que no había integridad referencial, con el consecuente riesgo de generar registros huérfanos y de empeorar el rendimiento en las consultas. Además, en la documentación del equipo de desarrollo anterior tampoco aparecía esta segunda relación de la tabla `users` con `activities` referente al profesor supervisor. Aunque el diseño actual no causaba problemas, confiar la integridad referencial exclusivamente a la aplicación no es adecuado ni seguro, por lo que se ha procedido a su corrección añadiendo la restricción en la migración correspondiente

Modelo Entidad-Relación (MER) resultante



Modelo Relacional (MR) resultante

```
Users(#user_id, first_name, last_name, email, hashed_password, first_log, user_type, created_at)

Materials(#material_id, name, description, image_path)
Activities(#activity_id, ##user_id, ##teacher_id, title, created_at)

Material_Activity(# ##activity_id, # ##material_id, units)

Storages(#storage, # ##material_id, qr_path)
Storage_Use(# ##material_id, # ##storage, units, min_units, cabinet, shelf, drawer)
Storage_Reserve(# ##material_id, # ##storage, units, min_units, cabinet, shelf)
Storage_Assignments(# ##material_id, # ##storage, #storage_type)
    * Tabla discriminante

Modifications(#action_datetime, # ##user_id, # ##material_id, # ##storage, # ##storage_type, units)
    * Entidad Asociativa
```

Mejoras principales propuestas: resultados

CAPTURAS DE JAVI!!

Instrucciones de despliegue

Entorno local (Windows)

Necesitamos un entorno MySQL que cuente con servidor y cliente. En nuestro caso utilizaremos WampServer, el cual ya incluye todo el software necesario para ejecutar y gestionar bases de datos

Se puede descargar WampServer en el siguiente enlace: <https://www.wampserver.com/en/>

Según la documentación del proyecto, la aplicación ha sido desarrollada bajo la versión menor PHP 7.4 con Laravel 8. Por lo tanto, debemos asegurarnos de contar con cualquier subversión o parche de dicha rama (como PHP 7.4.33)

Si nuestro entorno de WampServer no ha incluido alguna, podemos elegir una subversión y añadirla fácilmente como un add-on descargando el instalador ejecutable (.exe) desde su repositorio oficial de componentes, el cual automatiza todo el proceso:

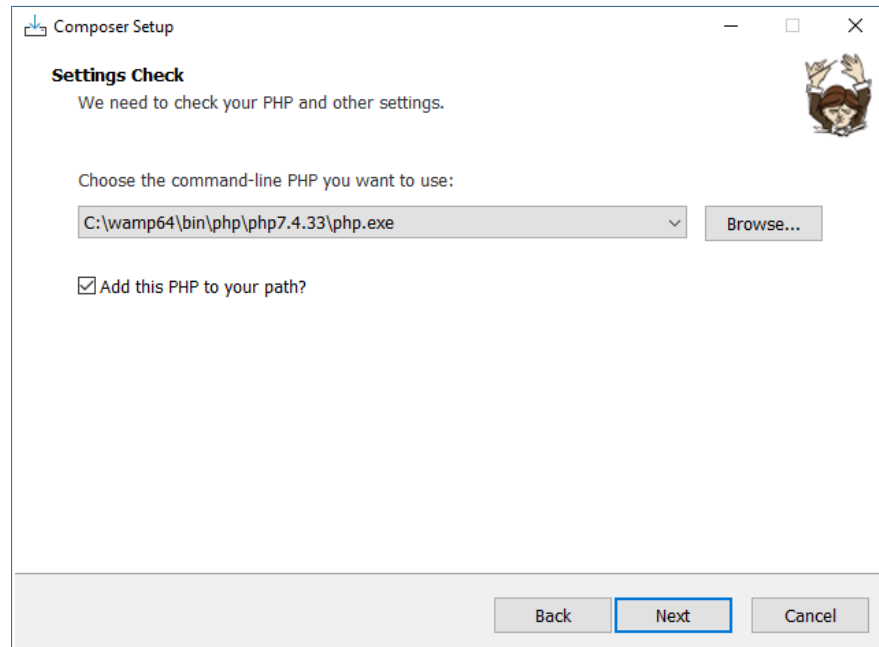
<https://wampserver.aviatechno.net/>

Para descargar todas las dependencias del proyecto (incluyendo Laravel 8), necesitamos instalar Composer, el gestor de dependencias estándar de PHP

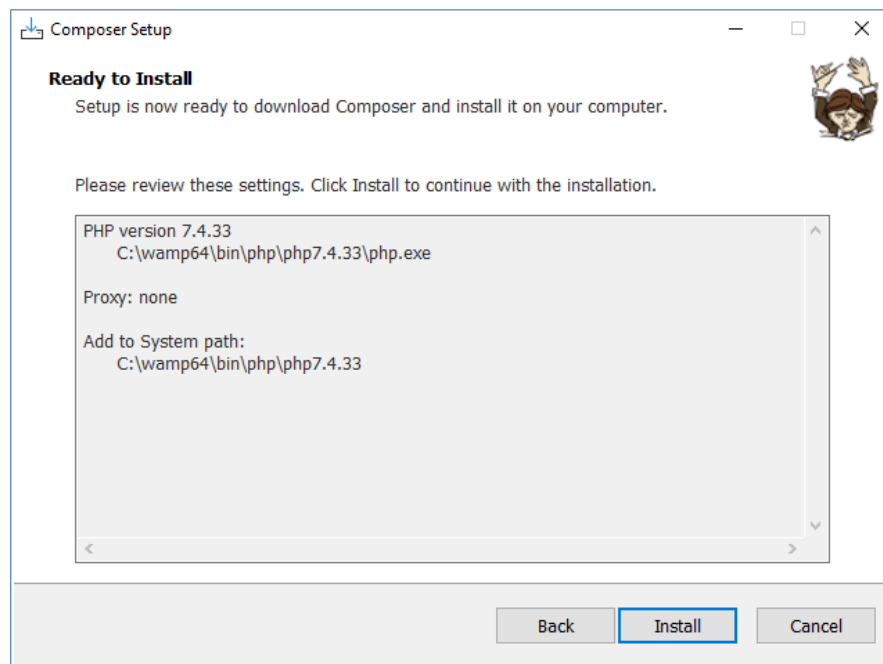
Se puede descargar Composer en el siguiente enlace: <https://getcomposer.org/download/>. Ahí, hacemos clic sobre el hipervínculo [Composer-Setup.exe](#) para descargar el instalador

Durante la instalación, debemos asegurarnos de marcar la opción **"Add this PHP to your path?"**, seleccionando la ruta de nuestra subversión de PHP 7.4. Esto es crítico porque Composer utilizará la primera versión de PHP que detecte en el PATH de Windows; al activar esta casilla, antepone la ruta seleccionada sobre cualquier otra versión instalada

Continuamos la instalación dejando las opciones por defecto/recomendadas en el resto de pasos



El resumen de la instalación debe quedar así



Después de la instalación, abrimos el CMD y escribimos el siguiente comando para comprobar que la versión de PHP que está usando Composer es la esperada

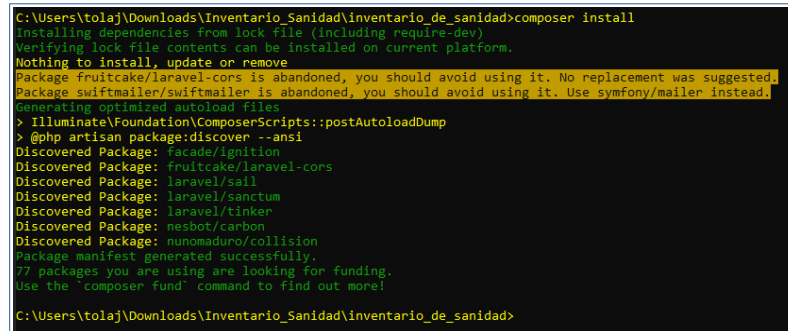
```
composer -V
```

Si todo está bien, procedemos a clonar nuestra bifurcación y accedemos al directorio raíz del proyecto Laravel

```
git clone https://github.com/franciscolopezl8n-ops/Inventario\_Sanidad  
cd Inventario_Sanidad/inventario_de_sanidad
```

Descargamos todas las dependencias del proyecto (listadas en el archivo `composer.json`)

```
composer install
```



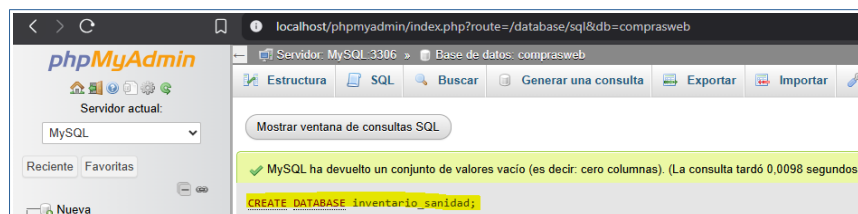
```
C:\Users\tolaj\Downloads\Inventario_Sanidad\inventario_de_sanidad>composer install  
Installing dependencies from lock file (including require-dev)  
Verifying lock file contents can be installed on current platform.  
Nothing to install, update or remove  
Package fruitcake/laravel-cors is abandoned, you should avoid using it. No replacement was suggested.  
Package swiftmailer/swiftmailer is abandoned, you should avoid using it. Use symfony/mailer instead.  
Generating optimized autoload files  
> Illuminate\Foundation\ComposerScripts::postAutoloadDump  
> @php artisan package:discover --ansi  
Discovered Package: facade/ignition  
Discovered Package: fruitcake/laravel-cors  
Discovered Package: laravel/sail  
Discovered Package: laravel/sanctum  
Discovered Package: laravel/tinker  
Discovered Package: nesbot/carbon  
Discovered Package: nunomaduro/collision  
Package manifest generated successfully.  
17 packages you are using are looking for funding.  
Use the "composer fund" command to find out more!  
C:\Users\tolaj\Downloads\Inventario_Sanidad\inventario_de_sanidad>
```

Creamos el archivo `.env` de Laravel. Este archivo contiene las variables de configuración del entorno (como los parámetros de conexión a la base de datos), las cuales varían según el entorno donde se ejecute la aplicación (por ejemplo, local o producción). Debido a que almacena información sensible, este archivo nunca debe subirse al repositorio. En su lugar, en cada nueva instalación se debe generar a partir del archivo plantilla `.env.example`, el cuál hemos adaptado a las necesidades específicas del proyecto

```
copy .env.example .env
```

En este archivo sugerimos cambiar las variables `DB_USERNAME` y `DB_PASSWORD`

Iniciamos el servidor MySQL y tomando como referencia el archivo `.env`, accedemos al cliente utilizando las credenciales configuradas y creamos la base de datos especificada en la variable `DB_DATABASE`. En nuestro caso, al haber instalado WampServer, iniciamos sus servicios y utilizamos el cliente phpMyAdmin incluido en la instalación



Regresamos al CMD y con los siguientes comandos inicializamos el entorno Laravel

```
php artisan key:generate
php artisan migrate --seed --force
php artisan storage:link
php artisan optimize:clear
php artisan cache:clear
```

Con esto, el entorno está preparado para correr la aplicación. Iniciamos el servidor web que Laravel incluye para el desarrollo, que es con el que nosotros hemos trabajado. Debemos asegurarnos de que sirva en un puerto alternativo para que no entre en conflicto con el servidor Apache de WampServer, donde se está ejecutando phpMyAdmin

```
php artisan serve
```

```
C:\Users\tolaj\Downloads\Inventario_Sanidad\inventario_de_sanidad>php artisan serve
Starting Laravel development server: http://127.0.0.1:8000
[Wed Mar 25 21:45:49 2026] PHP 7.4.33 Development Server (http://127.0.0.1:8000) started
```

Si accedemos a la URL que nos devuelve, accederemos a la página de inicio de sesión de la aplicación

Para acceder al panel de administración, necesitaremos iniciar sesión con un usuario de tipo administrador. Podemos encontrar todas las cuentas iniciales y sus credenciales en el seeder de usuarios `/database/seeder/UserSeeder.php`



Entorno local con Ngrok (Windows)

Hemos configurado el proyecto para que sea compatible con Ngrok, una herramienta que permite exponer el entorno local a Internet sin necesidad de desplegar un servidor externo o configurar nuestro router. Optamos por esta solución con la intención de mostrar los avances al cliente de forma provisional mientras buscábamos un alojamiento gratuito definitivo. Para obtener más información sobre el funcionamiento de Ngrok, se puede consultar el apartado de Despliegue en la bibliografía

Para habilitar esta funcionalidad en el entorno local que hemos preparado en la sección anterior, instalamos y configuramos Ngrok

- Se puede descargar Ngrok en el siguiente enlace: <https://ngrok.com/download/windows>
- Instrucciones oficiales para instalar y configurar Ngrok: <https://dashboard.ngrok.com/get-started/setup/windows>

Ejecutamos Ngrok mediante el siguiente comando, asegurándonos de especificar el mismo puerto local que utilizamos para servir la aplicación con el servidor de desarrollo de Laravel para que Ngrok pueda redirigir el tráfico externo correctamente hacia nuestra aplicación

```
ngrok http 8000
```

La terminal mostrará nuestro Dev Domain (URL pública en HTTPS), el cual servirá como puente de acceso desde la web hacia el puerto local expuesto. Debemos anotar esta dirección para el paso a continuación. La plataforma restringe las cuentas gratuitas a un único Dev Domain

En el directorio del proyecto Laravel, creamos un nuevo archivo de configuración de entorno `.env.ngrok` utilizando la plantilla

```
copy .env.example .env.ngrok
```

Nos aseguramos de que las siguientes variables tengan estos valores:

```
APP_URL=**DEV DOMAIN DE NGROK**
APP_ENV=ngrok
APP_DEBUG=false
APP_KEY=**MISMA CLAVE QUE EL ARCHIVO .env**
```

Ejecutamos el siguiente comando para limpiar el estado del framework y forzar lecturas de los archivos de configuración


```
php artisan config:clear && php artisan cache:clear && php artisan view:clear
```

Con esto, el entorno está preparado para exponer la aplicación a Internet de forma provisional

Iniciamos el servidor local de Laravel especificando las variables del nuevo entorno. Debemos asegurarnos de que sirva en el mismo puerto alternativo que está exponiendo Ngrok

```
php artisan serve --env=ngrok
```

Para ejecutar migraciones y seeders también es necesario especificar las variables de este entorno

```
php artisan migrate:fresh --seed --env=ngrok
```

Entorno de producción

ENLACE DE JAVI

Observaciones y mejoras a futuro

- Los scripts JS de la vista "materials.update.index" son un DESASTRE. Están llenos de código que no se usa, se usan formularios GET de forma innecesaria añadiendo param queries basura, y se obtienen los datos de los materiales a través de un fetch a un endpoint JSON del backend, dentro de la vista, cuando lo normal en Laravel es obtener los datos en el controlador para pasarlo a la vista. Aunque no creo que nos de tiempo, es muy recomendable un refactor, pues está muy sucio, puede ser frágil, añade sobreingeniería y es difícil de mantener
- En el frontend (JS), se abusa mucho de variables globales e interdependencias de entidades entre scripts, muchas de las cuáles son implícitas. Esto hace que el código sea muy frágil a cambios y difícil de mantener y sincronizar. También se usa el diseño de endpoints JSON por toda la aplicación, lo que me parece sobreingeniería: eso es algo que debería hacer el controlador, no la vista con fetches
- funciones JS duplicadas, sin centralización
- Implementar la vista de gestión de almacenamiento en el panel de profesores también?
- Mejorar notificaciones: - mejorar las notificaciones, usando notificaciones de laravel. al pulsar, nos lleva directamente al panel de actualización del stock
 - en algún momento se querrá sacar un material de un almacén en concreto o añadir un material a un almacén en el que no estaba? actualmente no está implementado
- cambiar de arquitectura. La lógica de presentación está muy acomodada a la vista que se renderiza con los datos provenientes de los controladores. Propongo api rests

Conclusiones

Bibliografía y recursos utilizados

Agentes de IA

- ChatGPT: <https://chatgpt.com/>
- Claude: <http://claude.ai/>
- Gemini: <https://gemini.google.com/>

Documentación de PHP

- PHP Manual: <https://www.php.net/manual/es/>
- W3School: <https://www.w3schools.com/php/>

Documentación de JavaScript

- MDN Web Docs: <https://developer.mozilla.org/es/docs/Web/JavaScript>
- W3School: <https://www.w3schools.com/js/>
- Fetch: https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Documentación de CSS

- MDN Web Docs: <https://developer.mozilla.org/es/docs/Web/CSS>
- W3School: <https://www.w3schools.com/css/>
- Librería de iconos: <https://docs.fontawesome.com/>

Despliegue

- Configurar versión de PHP que usará Composer: <https://github.com/composer/composer/issues/6277>
- Tutorial de Ngrok: <https://youtu.be/iAgJ6eCgUIA?si=6SQ8PITR9Be2gU3j>
- Instalación de Ngrok en Windows: <https://dashboard.ngrok.com/get-started/setup/windows>
- Qué es Ngrok: <https://en.wikipedia.org/wiki/Ngrok>

Tutoriales y cursos de Laravel:

- Curso de Laravel 8: <https://www.youtube.com/playlist?list=PLd3a4dr8oUsDAjQa8T0eKSyOxUCoiMVxO>
- Curso de Laravel desde cero: https://youtu.be/rn2LCOeNPds?si=j38w0ZSaa6aaJ-R_
- Envío de correos: <https://youtu.be/lqTOTk5FVTU?si=xZYIR-Ayc5sfyGct>
- Mailables: https://youtu.be/PuL86U2_kPE?si=4JOmtDDwkh61KLnt

Documentación de Laravel y lecturas:

- Documentación de Laravel: <https://laravel.com/docs>
- Sobre el soporte de claves compuestas en Eloquent: <https://blog.maqe.com/solved-eloquent-doesnt-support-composite-primary-keys-62b740120f>
- Mailables: <https://readouble.com/laravel/8.x/en/mail.html>

- Uso de múltiples archivos de entorno: <https://laravel-news.com/additional-env-files-in-laravel>
- Configuración en Laravel: <https://laravel-docs.com/es/docs/10.x/configuration>
- Intended: <https://dokov.bg/laravel-intended-redirect>
- Intended: <https://laraveldaily.com/post/auth-after-registration-redirect-to-previous-intended-page>
- Middlewares: <https://saniyathossain.medium.com/implementing-basic-authentication-middleware-in-laravel-and-lumen-dbc9125530d6>
- Middlewares: <https://laravel.com/docs/13.x/middleware>
- Protección CSRF: <https://laravel.com/docs/13.x/csrf#:~:text=Laravel%20automatically%20generates%20a%20CSRF,the%20requests%20to%20the%20application.>
- Discos y almacenamiento: <https://laravel.com/docs/13.x/filesystem>
- Relaciones Eloquent: <https://kinsta.com/es/blog/laravel-relaciones/>
- Lazy Loading VS Eager Loading: <https://styde.net/lazy-loading-vs-eager-loading/>
- Query Builder: <https://laravel.com/docs/13.x/queries>
- Autenticación y sesiones: <https://laravel.com/docs/13.x/authentication>
- Cookie: <https://api.laravel.com/docs/9.x/Illuminate/Support/Facades/Cookie.html>
- Patrón MVC: <https://developer.mozilla.org/es/docs/Glossary/MVC>

Documentación de librerías y clases:

- Simple QRCode: <https://github.com/SimpleSoftwareIO/simple-qrcode/tree/develop/docs/es>
- ZipArchive: <https://www.php.net/manual/en/class.ziparchive.php>

Bases de Datos y modelado

- Sobre las jerarquías solapadas: <https://es.scribd.com/document/850218250/DBMS-8>
- Sobre los discriminantes en jerarquías solapadas: <https://es.scribd.com/document/870319163/Week-7-Advanced-Data-Modeling-d6deb5df0e9530cae7dc058d643c5853>
- InnoDB: <https://www.ionos.es/digitalguide/hosting/cuestiones-tecnicas/que-es-innodb/>
- Sobre las entidades asociativas: <https://www.dataprix.com/bases-de-datos-master-de-software-libre-de-la-uoc/222-entidades-asociativas>
- Libro “Bases de datos” de Antonio Ahijado Sánchez (Marcombo, 2023)